Identificación de producto y estimación de ángulo con aprendizaje automático clásico, y un asistente agéntico para operar una línea de *pick & pack*

Sara Guerra

Prueba técnica de Ingeniero de IA — Indra Group

Informe técnico elaborado como entregable de una prueba técnica para Indra Group (octubre de 2026). No es una publicación arbitrada: el formato de artículo IEEE se usa como convención de presentación. Código, datos de evaluación y artefactos: github.com/hydemon8/prueba-tecnica-indra

***Resumen*—Este informe documenta la solución a una prueba técnica de Ingeniero de IA de Indra Group. El problema es sustituir la identificación y la alineación manual de productos en una línea de *pick & pack* por un sistema de visión que diga qué producto es, en qué ángulo está y si un brazo robótico puede actuar con seguridad. Sobre COIL-100, y con la restricción de usar aprendizaje automático clásico, un diagnóstico con vecino más cercano muestra que identificar el producto es fácil y que la dificultad está en el ángulo cuando faltan vistas. La identidad se resuelve con un histograma de color HSV y un bosque aleatorio, elegido entre 35 candidatos con un criterio fijado de antemano. El ángulo se estima con un lazo cerrado por objeto en un espacio de componentes principales, que además produce una distribución sobre el ángulo de la que salen dos medidas de incertidumbre. Con ellas, un control determinista deja actuar al robot en el 87,6 % de los casos con 0,7 % de acciones inseguras, frente al 4,1 % del método de referencia. Un asistente agéntico usa ese sistema como herramienta en siete tareas de operación, con permisos y aprobación humana aplicados en código, y completa la tarea en el 95 % de 174 ejecuciones de evaluación.**

***Palabras clave*—visión por computador, aprendizaje automático clásico, estimación de pose, espacio propio paramétrico, clasificación selectiva, agentes con herramientas, COIL-100.**

I. Introducción y contexto del problema

En una línea de *pick & pack*, un operario identifica cada producto y lo gira hasta una orientación fija para que un brazo robótico lo agarre. La prueba pide sustituir esa tarea por una cámara y un sistema que entregue, por cada imagen, cuatro salidas: el producto, su giro sobre el eje vertical con una medida de incertidumbre, las vistas de referencia más parecidas como evidencia, y la decisión de si el robot puede actuar. La cuarta es la que define el problema. Un robot que actúa con el producto equivocado empaca mal, y con el ángulo equivocado daña el producto o detiene la línea; el sistema debe saber abstenerse.

El enunciado fija tres condiciones. El modelo productivo debe ser de aprendizaje automático clásico, con características diseñadas y modelos estadísticos; un modelo profundo solo se admite como referencia de comparación. El robot tolera un error de ±10° en el ángulo. Y sobre el modelo debe construirse un asistente conversacional agéntico para el personal de planta, que lo use como herramienta y no lo reimplemente dentro de un *prompt*.

Los datos son COIL-100 [1]: 100 objetos fotografiados sobre un plato giratorio cada 5°, es decir, 72 vistas por objeto y 7.200 imágenes de 128 × 128 píxeles (Fig. 1). Son un sustituto controlado de las imágenes de una planta: fondo oscuro, objeto centrado e iluminación fija.

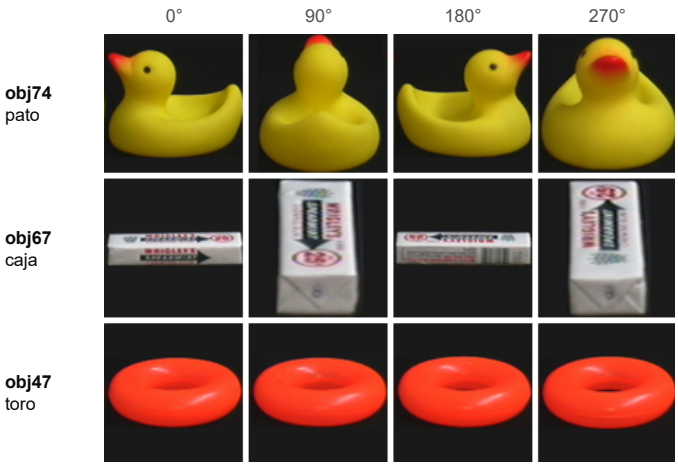


Fig. 1. Tres objetos de COIL-100 cada 90°, usados como ejemplo en el resto del documento: un pato (cada vista es distinta), una caja de chicles (se ve casi igual a 0° y a 180°) y un toro (se ve igual desde cualquier ángulo).

La solución separa dos planos (Fig. 2). En la ruta crítica, un servicio de visión determinista decide sobre cada objeto en unos 28 ms, sin intervención de un modelo de lenguaje. En el plano de supervisión, un asistente atiende a las personas y usa ese mismo servicio como herramienta. El documento sigue el orden en que se construyó: diagnóstico inicial (Sección II), características (III), protocolos y validación (IV), modelos de clasificación (V), modelo de ángulo (VI), asistente (VII y VIII), API y despliegue (IX) y conclusiones (X).

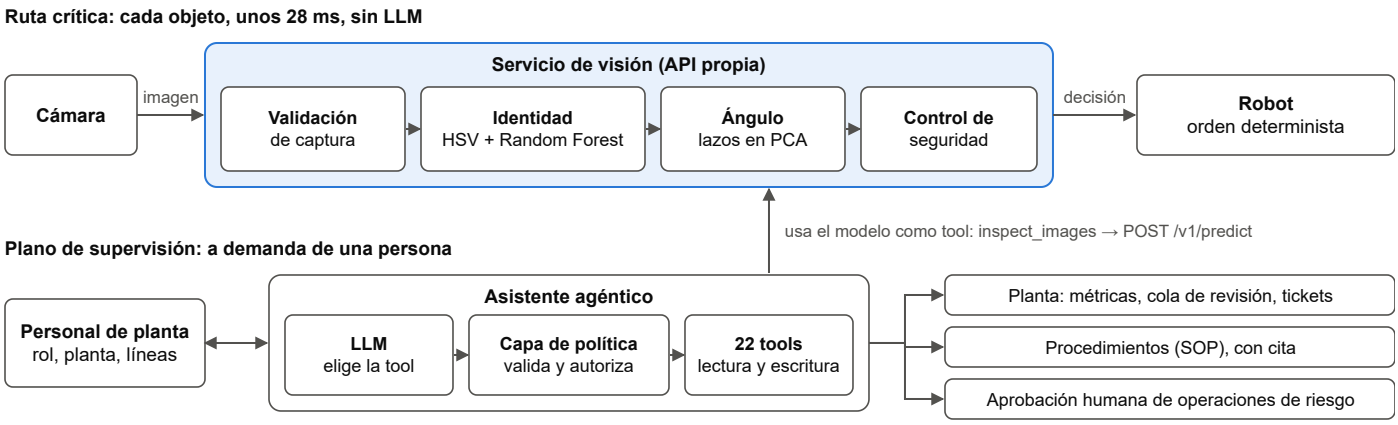


Fig. 2. Los dos planos de la solución. El servicio de visión decide sobre cada objeto sin modelo de lenguaje; el asistente lo usa como herramienta. El robot recibe una de cuatro órdenes: agarrar, segunda captura, revisión o rechazo.

II. Punto de partida: vecino más cercano

A. Método

Antes de diseñar nada se midió la dificultad con el método más simple disponible [12]. Cada imagen se reduce a 32 × 32 píxeles promediando bloques de 4 × 4 (*average pooling*), se aplana en un vector de 3.072 valores, se centra y se lleva a norma unitaria. La similitud entre dos imágenes es la distancia euclídea entre sus vectores, que con norma unitaria equivale a la similitud coseno. A una imagen nueva se le asigna el producto de su vecina más cercana. Para el ángulo se busca la vecina más cercana solo entre las vistas del producto correcto, de modo que la dificultad del ángulo se mide aparte de la de la identidad.

B. Resultados y limitaciones

La Fig. 3 resume el resultado. Con vistas de referencia cada 10°, el producto se identifica en el 99,97 % de los casos: clasificar no es la dificultad. El ángulo sí se degrada. Queda dentro de la tolerancia en el 97,8 % de los casos con vistas cada 10°, en el 76,0 % con doce vistas por producto y en el 28,8 % cuando la imagen cae en un arco de 60° que nunca se vio.

El método tiene cuatro límites que orientan el resto del diseño. Solo puede responder con un ángulo que ya vio, así que su error crece con el espaciado de las vistas. No extrapola: en el arco no visto, el 30,1 % de las estimaciones se equivoca en más de 90°. No indica cuánto confiar en su respuesta, de modo que no puede abstenerse. Y compara píxeles alineados, por lo que depende de que el objeto llegue centrado. La conclusión es que el problema está en estimar el ángulo cuando faltan vistas y en saber cuándo no responder. Este diagnóstico se hizo sobre las particiones de los protocolos; el método no tiene hiperparámetros y no se eligió ningún modelo con estas cifras.

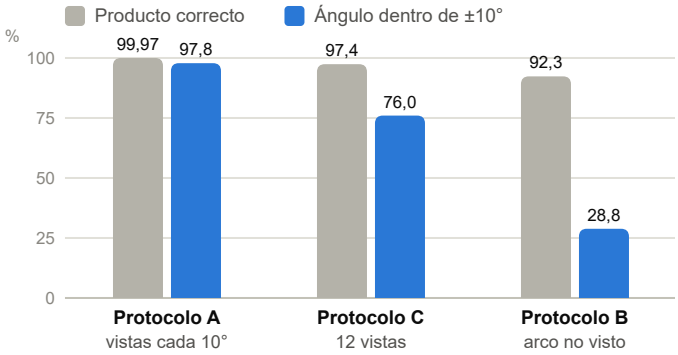


Fig. 3. Diagnóstico con vecino más cercano sobre píxeles de 32 × 32. La identidad se mantiene y el ángulo cae cuando faltan vistas.

III. Extracción de características

Un modelo clásico no opera sobre imágenes sino sobre vectores, y rinde según lo bien que esos vectores describan lo que importa. Se construyeron cuatro descriptores (Tabla I): los píxeles reducidos, un histograma de color en el espacio HSV, el histograma de gradientes orientados (HOG) [3] para la forma, y los patrones binarios locales (LBP) [4] para la textura. El histograma de color se calcula solo sobre los píxeles del objeto, separados del fondo con un umbral de brillo, y se le aplica la raíz cuadrada para comparar con la distancia de Hellinger.

Las combinaciones concatenan bloques normalizados a norma unitaria, para que cada familia pese lo mismo. Los bloques de más de 150 dimensiones se reducen a 150 con análisis de componentes principales (PCA) ajustado solo con las vistas de entrenamiento, y después se estandarizan.

La propiedad que organiza el diseño es que cada descriptor sirve para una tarea distinta. El color casi no cambia cuando el objeto gira: identifica bien el producto y no dice nada del ángulo. Los píxeles sí cambian con el giro, y por eso contienen el ángulo. De ahí salen dos modelos, uno para la identidad y otro para el ángulo, como en el trabajo original sobre este conjunto de datos [2].

TABLA I
DESCRIPTORES DE IMAGEN

Bloque	Definición	Dim.	Qué describe
Píxeles	RGB reducido a 32 × 32 con interpolación bilineal, en [0, 1]	3.072	Apariencia completa; cambia con el giro
Color HSV	Histograma de 16 × 4 × 4 intervalos sobre los píxeles del objeto, con raíz cuadrada	256	Color; casi no cambia con el giro
Forma HOG	Gris a 64 × 64, celdas de 8 × 8, bloques de 2 × 2 y 9 orientaciones	1.764	Forma, por sus bordes
Textura LBP	Patrones uniformes con (8, 1) y (16, 2) vecinos y radio	28	Textura

IV. Protocolos, validación y criterio de selección

A. Protocolos de evaluación

El enunciado define cuatro protocolos (Fig. 4). Un protocolo no es un modelo distinto: es una forma de repartir las vistas entre entrenamiento y prueba para medir una capacidad concreta. El sistema y el código son los mismos en los cuatro. El protocolo A mide la interpolación entre vistas conocidas (3.600 imágenes de entrenamiento y 3.600 de prueba). El B esconde un arco de 60° de cada objeto y pregunta por él (6.000 y 1.200). El C deja doce vistas por producto (1.200 y 6.000). El D excluye diez objetos por completo y mide si el sistema sabe decir que no los conoce (6.480 y 720). B y D dependen de un sorteo, así que se repiten con tres semillas y se reporta la media y la desviación.

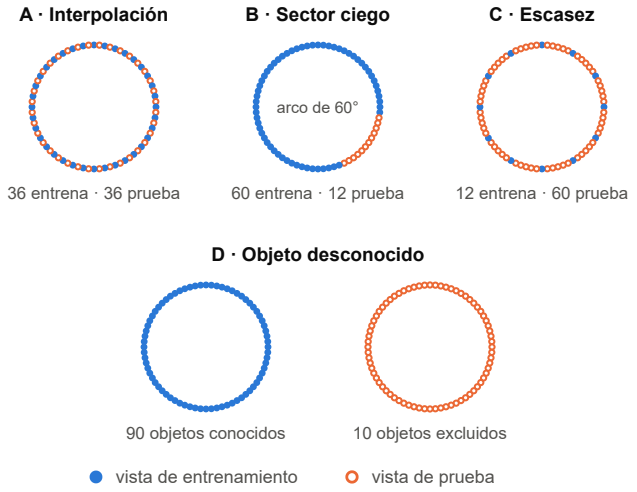


Fig. 4. Reparto de las 72 vistas de un objeto en cada protocolo. En B el arco se sortea por objeto; en D se excluyen objetos completos.

B. Validación interna sin fugas

Dos vistas separadas 5° son casi la misma imagen, de modo que una validación cruzada con particiones al azar filtraría información. Por eso la validación interna se construye solo con vistas de entrenamiento y por bloques de ángulo o de objeto, imitando cada protocolo (Tabla II). En A y C es más exigente que la prueba, lo que produce umbrales conservadores. La prueba de cada protocolo se evalúa una sola vez y ningún umbral se ajusta con ella.

TABLA II
VALIDACIÓN INTERNA DE CADA PROTOCOLO

Prot.	Particiones internas, solo con vistas de entrenamiento
A	Dos: entrena con 0°, 20°, 40°… y valida con 10°, 30°…, y al revés. Espaciado doble que en la prueba
B	Tres: se retira otro arco de 60° dentro de los 300° vistos
C	Dos: entrena con 0°, 60°… y valida con 30°, 90°…, y al revés. Huecos de 60°
D	Tres: diez objetos del catálogo hacen de desconocidos, distintos de los de la prueba

C. Búsqueda en rejilla con criterio fijado de antemano

La identidad se eligió con una búsqueda en rejilla sobre 35 candidatos: cinco representaciones (píxeles, color, forma, textura y la combinación de las tres últimas) por siete configuraciones de clasificador. Cada candidato se ajusta en las siete particiones internas de A, B y C. El criterio se escribió en el archivo de configuración antes de ejecutar ninguna selección:

1. Restricción de seguridad: en la validación interna de D, como máximo el 5 % de los objetos desconocidos puede aceptarse cuando se acepta el 95 % de los conocidos.
2. Entre los que la cumplen, gana la mayor exactitud en el peor caso entre A, B y C.
3. Si la diferencia es menor de 0,5 puntos, gana el de menor latencia.

La puntuación de novedad es la distancia a la vista de referencia más cercana en la representación del candidato. Se registra además el área bajo la curva de riesgo y cobertura [11], que mide si la confianza del clasificador ordena bien sus errores. La rejilla es pequeña a propósito: una exploración previa mostró que la representación pesa mucho más que el clasificador.

V. Modelos de clasificación: selección y resultados

A. Clasificadores

Se compararon cuatro familias, con scikit-learn [10]: vecino más cercano; regresión logística multinomial con regularización $C \in \{1, 10\}$; máquina de vectores de soporte con núcleo gaussiano [6] y $C \in \{1, 10, 100\}$; y bosque aleatorio [5] de 500 árboles. La confianza de la máquina de vectores de soporte se obtiene con una función *softmax* sobre sus puntuaciones. La calibración habitual por validación cruzada bajaba la exactitud interna de C al 75,7 %, porque sus particiones sin barajar eran, de hecho, sectores ciegos.

B. Selección

La Fig. 5 muestra el peor caso de los 35 candidatos. La representación domina: el mejor candidato sobre píxeles (78,2 %) queda veinte puntos por debajo del mejor sobre color (98,5 %), mientras que con el mismo descriptor de color los

clasificadores difieren en unos cinco puntos. La forma y la textura por separado incumplen la restricción de novedad, con 15,5 % y 65,4 % de desconocidos aceptados.

Ganó el histograma de color con bosque aleatorio: mejor peor caso (98,50 %, en C), 1,02 % de desconocidos aceptados y la mejor curva de riesgo y cobertura de toda la rejilla. La hipótesis previa era que ganaría la combinación de color, forma y textura, que quedó segunda con regresión logística (97,00 %). Se respetó el criterio.

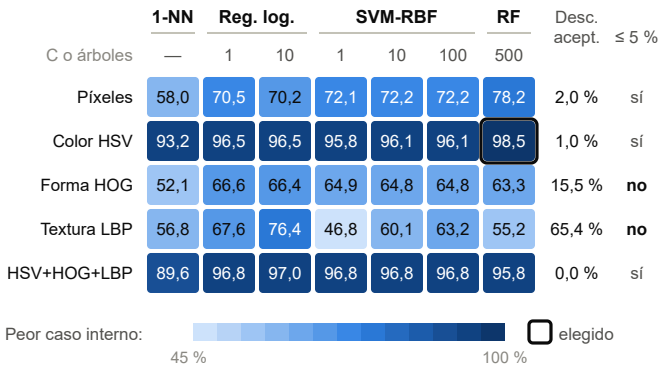


Fig. 5. Exactitud del peor caso entre las validaciones internas de A, B y C para los 35 candidatos. La columna de la derecha indica si la representación cumple la restricción de novedad.

C. Resultados en prueba

La Tabla III compara el modelo elegido con un método de referencia simple (PCA de píxeles con vecino más cercano, sin rechazo) y con una referencia moderna: DINOv2 [7] como extractor de características, con regresión logística. El modelo clásico mantiene la identidad por encima del 99 % en los tres protocolos y rechaza el 99,8 ± 0,3 % de los objetos desconocidos.

TABLA III
IDENTIDAD EN PRUEBA (%)

Medida	Referencia simple	Propuesto	DINOv2
Clase, protocolo A	99,8	99,9	100,0
Clase, protocolo B	84,2 ± 0,8	99,0 ± 0,4	99,8 ± 0,3
Clase, protocolo C	94,2	99,6	100,0
Desconocidos rechazados, D	0,0	99,8 ± 0,3	93,1 ± 2,7

D. Análisis

Los errores de identidad se concentran en C y en pocos pares: piezas de madera del mismo tono, como los objetos 51, 11 y 12. Es coherente con un modelo que se apoya en el color. La referencia moderna mejora la identidad menos de un punto, rechaza peor los desconocidos (93,1 ± 2,7 %) y cuesta más: su extractor tiene unos 22 millones de parámetros y se ejecutó en GPU.

La debilidad del modelo elegido también viene del color. En pruebas posteriores sobre imágenes alteradas, un aumento del brillo del 15 % hizo que el 66 % del producto conocido se rechazara como desconocido. Es un fallo de disponibilidad, no de seguridad, pero detendría una línea. El segundo candidato del criterio rechazó solo el 0,5 % en esa condición, lo que sugiere añadir un término de robustez al criterio de selección.

VI. Modelo de lazos para el ángulo

A. Idea

El objeto solo puede girar sobre el eje vertical, así que su apariencia depende de un único parámetro. Si cada imagen se representa como un punto, al girar el objeto ese punto se mueve de forma continua y vuelve al inicio al completar la vuelta. Las vistas de un objeto forman una curva cerrada, un lazo. Es el espacio propio paramétrico de Murase y Nayar [2].

B. Construcción

La Fig. 6 resume los pasos. La imagen se reduce a 32 × 32 píxeles y se aplana en 3.072 valores, la misma descripción del diagnóstico inicial. Una PCA global, ajustada una vez con las vistas de entrenamiento de todos los objetos, la comprime a 50 componentes, que conservan el 88,6 % de la varianza. Después, para cada objeto se toman los puntos de sus vistas conocidas, ordenados por ángulo, y se unen con un *spline* cúbico periódico, componente a componente. La curva se evalúa cada 0,5°, lo que da 720 puntos por objeto: la apariencia esperada en cada ángulo, también en los que no se fotografiaron. El ángulo estimado de una imagen nueva es el del punto más cercano del lazo de su producto.

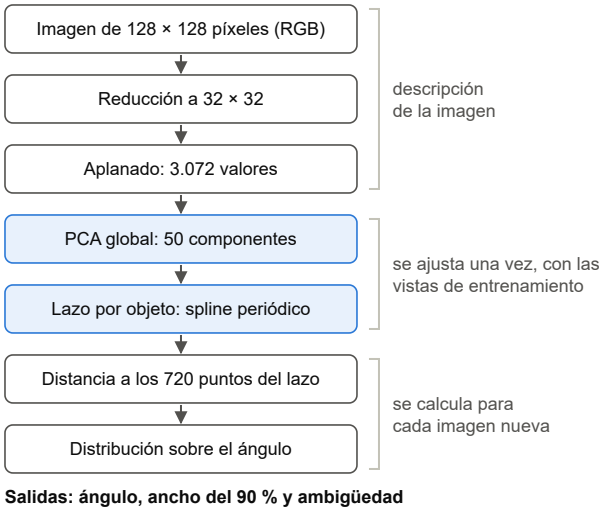


Fig. 6. Del píxel al ángulo. La PCA y los lazos se ajustan una vez; la distancia y la distribución se calculan para cada imagen.

C. Forma del lazo y simetría

La forma del lazo anticipa si el ángulo se puede estimar (Fig. 7). En un objeto sin simetría cada ángulo ocupa un lugar distinto. En uno que se ve igual de frente que de espaldas, las vistas opuestas coinciden y el lazo se recorre dos veces. La simetría de media vuelta se mide con la razón

$$r = \frac{\operatorname{med}_\theta \, d^2(\theta, \theta + 180°)}{\operatorname{med}_\theta \, d^2(\theta, \theta + 10°)} \qquad (1)$$

donde d^2 es la distancia al cuadrado entre dos puntos del lazo. Si $r < 1$, la vista opuesta está más cerca que la vecina; ocurre en diez de los cien objetos. La razón es relativa y no ve el tamaño del lazo. El toro (obj47) tiene $r = 14{,}67$, pero su lazo mide 0,27 frente a una mediana de 15,3: es el tercero más pequeño. Se ve igual desde cualquier ángulo y su ángulo no es observable. Son dos simetrías distintas y se detectan con medidas distintas.

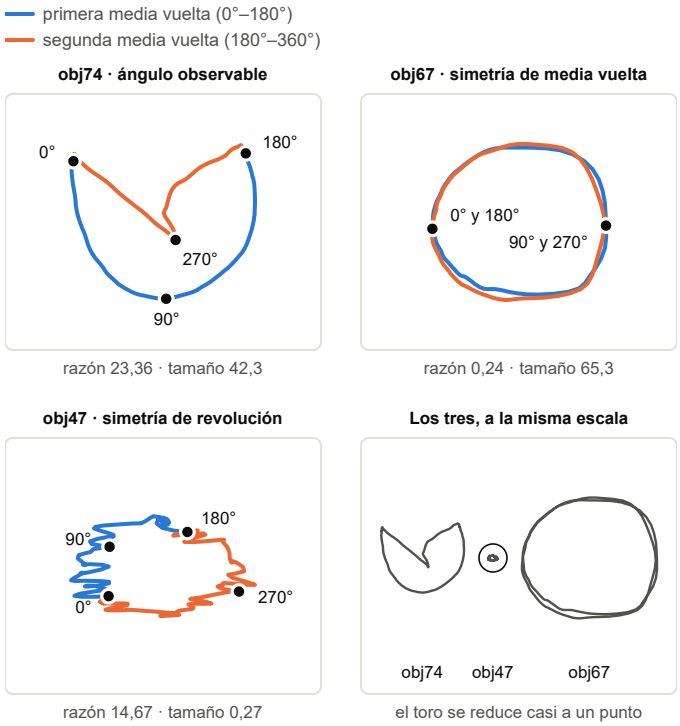


Fig. 7. Lazos de tres objetos, cada uno en su plano principal (las dos direcciones en que más varía, que reúnen entre el 67 % y el 80 % de su varianza). En el pato cada ángulo ocupa un lugar; en la caja las dos medias vueltas coinciden; el toro, a escala real, es casi un punto. «Razón» es r en (1); «tamaño» es la distancia al cuadrado mediana al centro del lazo.

D. Incertidumbre a partir de una distribución

Un ángulo sin una medida de confianza no permite abstenerse. Se supone que la imagen observada es un punto del lazo más ruido gaussiano, y que todos los ángulos son igual de probables a priori. La probabilidad de cada ángulo θ_k de la rejilla, dada la imagen, es entonces

$$p(\theta_k) \propto \exp\!\left(-\frac{d_k^{\,2} - d^2_{\text{mín}}}{s\,\tau_0}\right) \tag{2}$$

donde d_k es la distancia de la imagen al punto k del lazo, τ_0 es la distancia al cuadrado mediana entre vistas consecutivas, que fija la escala del ruido, y s se calibra en validación interna para que el conjunto creíble del 90 % contenga el ángulo real el 90 % de las veces ($s = 0{,}040$ y cobertura del 90,6 % en A). De la distribución salen dos medidas. El *ancho* es el tamaño, en grados, del menor conjunto de ángulos que reúne el 90 % de la probabilidad. La *ambigüedad* compara el mejor ángulo con el mejor candidato situado a más de 20° de él:

$$a = \left(\frac{d^2_{\text{mín}} + \varepsilon}{d^2_{\text{lejos}} + \varepsilon}\right)^{1/2} \tag{3}$$

Si a se acerca a 1, hay un segundo ángulo lejano casi tan bueno como el primero. La Fig. 8 muestra los tres comportamientos con vistas reales de prueba. En el pato, el perfil de distancia tiene un único valle y la distribución un único pico (ancho de 3,5°). En el toro el perfil es casi plano y la probabilidad se reparte (ancho de 103,5°). En la caja de chicles, a 105°, hay dos valles de profundidad casi igual y dos picos; el modelo elige el equivocado y estima 285°, con 180° de error y un ancho de solo 1,5°. El ancho lo aceptaría; la ambigüedad (1,00) lo detiene.

Ese caso justifica la segunda medida. La primera versión del control usaba solo el ancho, y en la validación interna de A dejaba pasar un 1,05 % de casos con más de 10° de error. Con el umbral de ambigüedad, calibrado en esa misma validación, bajó al 0,47 %. Conviene precisar el alcance: con 50 componentes, la mayoría de las vistas opuestas de un objeto simétrico sí se distinguen. De las 36 vistas de prueba de la caja de chicles, solo esa supera el umbral; las demás se estiman bien.

E. Control de seguridad

La decisión la toman cuatro reglas deterministas, en orden (Tabla IV). Todos los umbrales se calibran en validación interna. La tolerancia de ±10° se traduce en un objetivo de riesgo: que al menos el 99,5 % de las veces que el robot actúa, el ángulo esté dentro de la tolerancia. El ancho máximo es de 20°, el doble de la tolerancia, y el umbral de ambigüedad es el mayor que cumple ese objetivo entre los casos aceptados.

TABLA IV
REGLAS DEL CONTROL DE SEGURIDAD

Decisión	Condición	Orden al robot
Desconocido	Novedad por encima de su umbral	Carril de rechazo
Revisión	Confianza de clase baja, o la vista más parecida es de otro producto	Revisión humana
Ángulo no certificado	Hueco de más de 30° en la galería del producto, ancho mayor de 20° o ambigüedad alta	Segunda captura
Actuar	Ninguna de las anteriores	Agarrar, con producto y ángulo

F. Resultados

La Fig. 9 y la Tabla V resumen la prueba. El modelo de lazos supera al vecino más cercano en los tres protocolos, sobre todo con pocas vistas (95,0 % frente a 74,1 % en C). Con las características de DINOv2 el mismo método estima peor el ángulo, lo que es coherente con un extractor entrenado para no depender del punto de vista.

En A, el robot actúa en el 87,6 % de los casos y el 0,7 % de todas las imágenes termina en una acción insegura, frente al 4,1 % del método de referencia, que actúa siempre. El objetivo de riesgo se cumplió en validación interna (0,47 %) y no del todo en prueba (0,79 % de las acciones): la validación interna

usa un espaciado doble y no representa exactamente la prueba. En B el sistema no actúa, por la regla de cobertura, y con una segunda captura girando el objeto 180° recupera el ángulo en el 97,4 % de los casos y actúa en el 93,9 % con 0,72 % de acciones inseguras. En C acierta el ángulo pero no puede demostrar el criterio con doce vistas, y no actúa. En D rechaza el 99,8 ± 0,3 % de los objetos desconocidos.

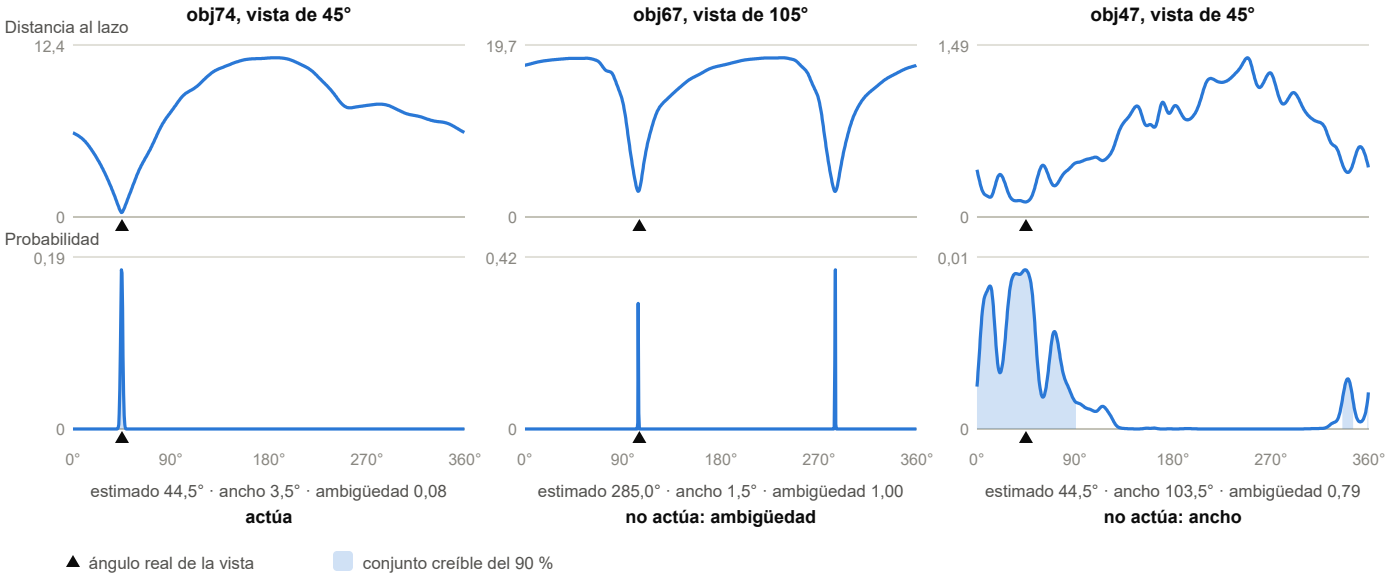


Fig. 8. Perfil de distancia al lazo (arriba) y distribución sobre el ángulo (abajo) para tres vistas de prueba del protocolo A. Cada panel tiene su propia escala vertical; la distancia máxima del toro es unas diez veces menor que la de los otros dos.

TABLA V
ÁNGULO Y DECISIÓN EN PRUEBA (%)

Prot.	Modelo	Ángulo ±10°	Actúa	Acciones inseguras
A	Referencia simple	96,1	100,0	4,1
A	**Propuesto**	97,0	87,6	0,7
A	DINOv2	95,3	75,1	0,4
B	Referencia simple	28,2 ± 0,1	100,0	72,7 ± 0,1
B	**Propuesto**	55,2 ± 1,6	0,0	0,0
B	DINOv2	21,8 ± 0,3	0,0	0,0
C	Referencia simple	74,1	100,0	28,3
C	**Propuesto**	95,0	0,0	0,0
C	DINOv2	87,7	0,0	0,0
D	Referencia simple	—	100,0	100,0
D	**Propuesto**	—	0,0	0,0
D	DINOv2	—	0,0 ± 0,1	0,0 ± 0,1

Media ± desviación de tres semillas en B y D. El ángulo se mide con la clase conocida. Una acción insegura es actuar con el producto equivocado o con el ángulo fuera de la tolerancia. La referencia simple no tiene control y actúa siempre.

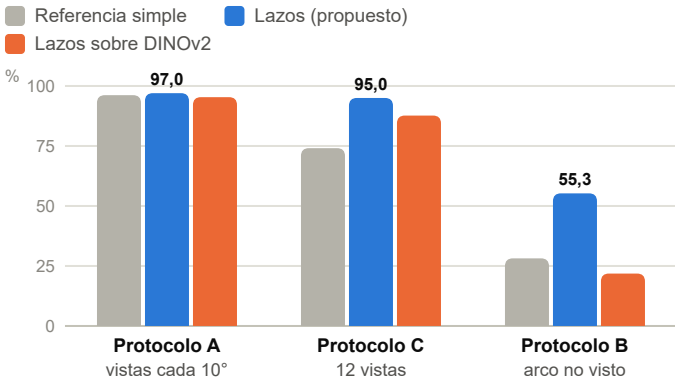


Fig. 9. Ángulo dentro de ±10° en prueba, con la clase conocida. La etiqueta corresponde al modelo propuesto.

VII. Asistente agéntico: planteamiento

A. Qué es agéntico y qué no

El asistente no está en la línea. La decisión sobre cada objeto la toma el servicio de visión, que es determinista; un modelo de lenguaje ahí añadiría segundos, costo por objeto y respuestas no repetibles en una decisión de seguridad. Lo agéntico se reserva para lo que un modelo de lenguaje hace bien: entender una petición abierta, elegir y encadenar herramientas, pedir lo que falta y explicar el resultado. Si el asistente o el modelo de lenguaje fallan, la línea sigue funcionando.

B. Patrón

Es un solo agente con herramientas (*tools*) que razona y actúa por turnos [8], bajo una capa de política determinista. Se descartaron tres alternativas. Un sistema multiagente multiplicaría los tokens por dos o tres, añadiría errores de enrutamiento y obligaría a replicar los permisos. Los flujos fijos sin agente son más baratos y predecibles, pero cada pregunta nueva exige código; se adoptaron solo para lo que debe ser fijo. Planificar primero y ejecutar después fija el plan antes de ver los datos, y en un diagnóstico el siguiente paso depende del resultado anterior.

C. Bucle y arnés

El bucle del agente y el código que lo rodea (el arnés, *harness*) están escritos a mano, sin bibliotecas de agentes (Fig. 10). En cada turno, el arnés arma el contexto, selecciona las *tools* que corresponden al rol de la persona y llama al modelo. Si el modelo responde con texto, el turno termina. Si pide una *tool*, la capa de política la valida y la ejecuta o la rechaza, y el resultado vuelve al modelo en JSON. El ciclo se repite hasta una respuesta o hasta un límite (Tabla VI). Al alcanzar un límite no se llama otra vez al modelo: se responde con un texto fijo que indica qué se completó. El modelo es Gemini, detrás de una interfaz de proveedor neutra. Un proveedor guionizado lo sustituye en las pruebas, para forzar comportamientos que un modelo real no produce a voluntad, como repetir la misma llamada sin fin.

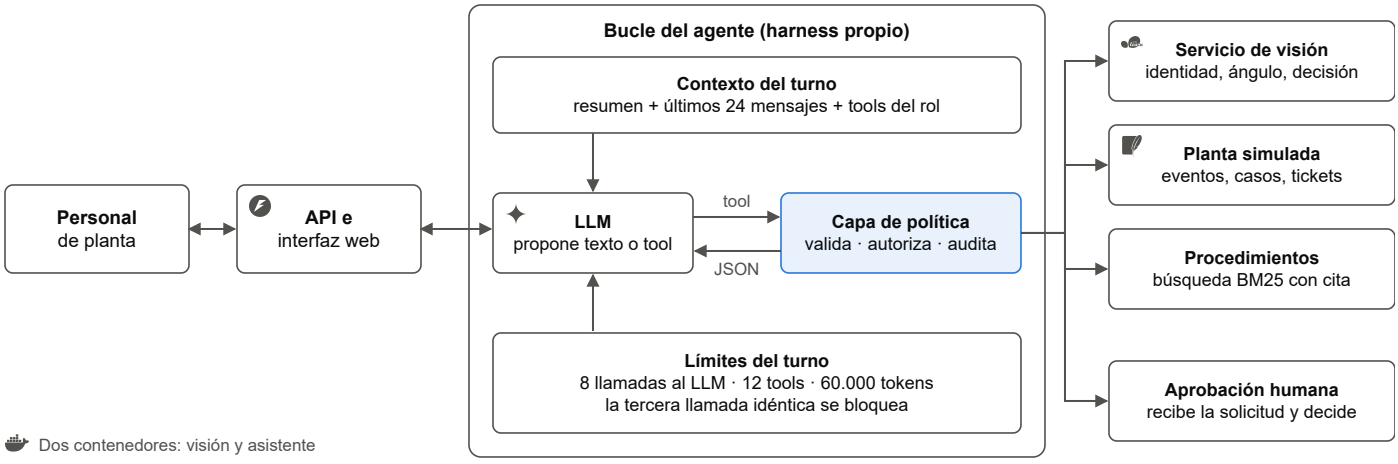


Fig. 10. Arquitectura del asistente. El modelo de lenguaje propone y la capa de política ejecuta. Las operaciones de riesgo terminan en una solicitud que decide otra persona.

TABLA VI
LÍMITES DE UN TURNO

Límite	Valor	Al alcanzarlo
Llamadas al modelo	8	Fin del turno
Tools ejecutadas	12	Fin del turno
Misma llamada con los mismos argumentos	2	La tercera se bloquea
Tokens	60.000	Fin del turno
Plazo del servicio de visión	4 s	Un reintento y error estructurado

D. Manejo del contexto y memoria

Se distinguen cuatro cosas. El estado de ejecución dura un turno. El historial conversacional se guarda por sesión. La memoria persistente, con casos, tickets y aprobaciones, vive en la base de datos de la planta y no es memoria del modelo: se consulta con *tools*, así que siempre está al día y pasa por los mismos permisos. El conocimiento recuperado son secciones de procedimientos, obtenidas con BM25 [9] y devueltas con su cita.

El contexto se mantiene acotado con cuatro reglas. Entre turnos solo viaja texto; los resultados de las *tools* y las imágenes no se reinyectan. Los últimos 24 mensajes van íntegros y los anteriores se pliegan en un resumen de 2.400 caracteres como máximo. Cada resultado de *tool* se trunca a 9.000 caracteres. Y el modelo solo recibe las *tools* de su rol: un operario ve 3 de 22 y una supervisora 18.

E. Políticas

La capa de política es la única vía de ejecución. Valida los argumentos contra el contrato de cada *tool*, autoriza por rol y por línea, y registra cada intento, también los rechazados, en una tabla de auditoría. La planta nunca es un argumento: sale de la sesión. El texto que devuelven las *tools*, como un ticket o un procedimiento, se trata como dato y no como instrucción.

Las cinco operaciones de riesgo (pausar o reanudar una línea, cambiar el umbral de confianza, enviar un ángulo manual al robot, promover una versión de modelo y pasar un producto a producción) no se ejecutan desde el asistente. La *tool* comprueba las precondiciones y crea una solicitud pendiente. La decide otra persona, con rol aprobador, desde un punto de acceso que no es una *tool*; quien solicita no puede aprobar, y las precondiciones se vuelven a comprobar al apro-

bar. El asistente no puede aprobarse a sí mismo porque aprobar no está a su alcance.

VIII. Capacidades e incorporación de los sistemas de aprendizaje automático

A. Capacidades

El asistente cubre los siete casos de uso del enunciado con 22 *tools*, once de lectura y once de escritura (Tabla VII). Atiende a supervisores, calidad, mantenimiento, ingeniería de modelos, gerencia y operarios, cada uno con las *tools* de su rol.

TABLA VII
Casos de uso y *tools* principales

Caso de uso	***Tools* principales**
Inspección de imagen†	`inspect_images`
Cola de revisión humana	`list_review_cases`, `resolve_review_case`, `add_to_retraining_dataset`
Consulta operativa†	`get_shift_metrics`, `get_line_timeline`, `get_camera_health`, `get_line_status`
Diagnóstico de falla	`get_grasp_failures`, `search_procedures`, `create_ticket`
Alta de producto nuevo†	`register_sku_candidate`, `request_sku_promotion`
Operación de riesgo	`set_line_state`, `set_confidence_threshold`, `send_manual_angle`, `promote_model_version`
Entrega de turno	`create_shift_handover`

† Usa el modelo de visión.

B. Cómo se incorpora el modelo de visión

Para el asistente, la clasificación y la estimación de ángulo no son dos sistemas sino una sola *tool*. `inspect_images` envía las imágenes que sube la persona al mismo servicio que usa el robot y devuelve, por imagen, la decisión del control con sus motivos, el producto y su confianza, la novedad, el ángulo y su ancho, la calidad de la captura, las vistas más parecidas y la acción recomendada, que es una tabla fija por decisión. El modelo de lenguaje solo recibe ese resultado; nunca los píxeles. Una regla del *prompt* establece que el producto y el ángulo los determina únicamente esa *tool*. Ante la imagen del toro, por ejemplo, el asistente informa que es el producto 47, que el ángulo no está certificado porque su ancho es de 51,5°, y propone una segunda captura. Si el servicio de visión falla, hay un reintento y después un error estructurado: el asistente dice que no pudo analizar la imagen y no inventa un resultado.

El segundo punto de entrada es el alta de un producto nuevo. `register_sku_candidate` comprueba que haya al menos 36 vistas y ningún hueco de más de 30°, las condiciones que el modelo de ángulo necesita para certificar. Después pasa cada vista por el servicio de visión, para confirmar que las capturas son válidas y que el producto no se confunde con uno del catálogo. Si lo supera, queda registrado en un entorno de prueba; el paso a producción es una operación de riesgo.

El tercero es de solo lectura. El servicio de visión vigila, por línea, la tasa de objetos desconocidos, y activa una alerta de captura si sube de forma brusca; mientras dura, los desconocidos se retienen para revisión en lugar de rechazarse. `get_line_status` incluye esa alerta, con su causa probable, y la interfaz la muestra en un panel de líneas. En esta demostración, la planta es simulada y la evaluación de desempeño del producto nuevo también, porque no se reentrena el modelo. El servicio de visión es real.

C. Evaluación

El asistente se evaluó con 45 casos reproducibles, 19 de ellos adversos: imágenes fuera de dominio, resultados no confiables, argumentos inválidos, caídas del servicio de visión, bucles e intentos de saltarse los permisos. Cada caso se ejecuta sobre una planta recién creada y lo juzga código, no otro modelo: qué *tools* se llamaron y con qué argumentos, el estado final de la base de datos y el contenido de la respuesta. Hubo 174 ejecuciones válidas, entre tres y cinco por caso.

La tarea se completó en el 95 % de las ejecuciones (166 de 174), con la *tool* correcta en el 98 % y argumentos correctos en el 100 % (Fig. 11). En los casos adversos el éxito fue del 93 %. Ninguna ejecución cambió indebidamente el estado de la planta: los fallos son de contenido de la respuesta. El peor resultado está en el diagnóstico, con un fallo sistemático ante la pregunta por un procedimiento que no existe. Las cifras son optimistas, porque el *prompt* se ajustó con estos mismos casos y no hay un conjunto reservado.

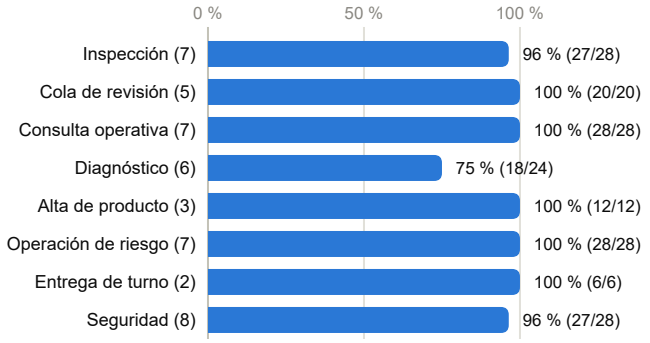


Fig. 11. Éxito de la tarea por categoría, sobre las ejecuciones válidas. Entre paréntesis, el número de casos de cada categoría.

IX. API y despliegue

A. API del servicio de visión

El modelo se sirve con una API HTTP propia, independiente del asistente (Tabla VIII). El servicio publica su documentación interactiva en `/docs`, y el contrato OpenAPI está versionado en el repositorio (`doc/openapi/vision.json`).

TABLA VIII
API DEL SERVICIO DE VISIÓN

Método y ruta	Qué hace
`POST /v1/predict`	Una o varias imágenes; devuelve decisión, orden para el robot, identidad, ángulo y vistas similares
`GET /v1/model`	Versión del modelo y de los datos, huella del artefacto y umbrales del control
`GET /v1/catalog`	Productos del catálogo: vistas de referencia, hueco angular máximo y simetría
`GET /v1/reference/{producto}/{ángulo}`	Vista de referencia en PNG, usada como evidencia
`GET /v1/lines/{línea}/monitor`	Estado de la alerta de captura de una línea
`GET /healthz` `GET /readyz`	Proceso vivo; modelo cargado y verificado
`GET|POST /admin/fault`	Simula errores y demoras. Solo si se habilita, para las pruebas de robustez

La operación principal es `POST /v1/predict`. Recibe una o varias imágenes PNG o JPEG en un formulario, en el campo `images`: hasta 16 por petición y 5 MB cada una. El parámetro `top_k` fija cuántas vistas similares se devuelven, entre 1 y 20. Hay tres cabeceras opcionales: `X-API-Key`, que solo se exige si el servicio se configura con una clave; `X-Line-ID`, que identifica la línea y activa su alerta de captura; y `X-Request-ID`, para seguir la petición. La respuesta trae un resultado por imagen (Tabla IX). A continuación, una llamada real con la respuesta abreviada:

```
$ curl -F "images=@obj5__40.png" \
    "http://127.0.0.1:8001/v1/predict?top_k=2"

{"request_id": "4c4e8e05…",
 "model_version": "4e39f878-rf",
 "results": [{
   "filename": "obj5__40.png",
   "decision": "ACT", "reasons": [],
   "robot_command": {"action": "PICK",
       "object_id": 5, "angle_deg": 40.0},
   "object_id": 5, "class_confidence": 0.974,
   "novelty": 0.0, "angle_deg": 40.0,
   "angle_credible_width_deg": 6.0,
   "quality": {"ok": true, "issues": [], …},
   "topk_pose": [{"object_id": 5,
       "angle_deg": 40.0, "delta_deg": 0.0}, …],
   "topk_appearance": [{"object_id": 5,
       "angle_deg": 40.0, "distance": 0.0}, …]}]}
```

Los errores comparten un mismo cuerpo, con `code`, `message` y `request_id`: 401 si falta la clave, 404 si no existe la vista de referencia, 413 si se supera el tamaño o el número de imágenes, 422 si un parámetro no es válido y 503 si el modelo no está cargado. Una imagen ilegible o vacía no es un error: devuelve la decisión `BAD_CAPTURE`, con la orden de repetir la captura.

TABLA IX
CAMPOS DE LA RESPUESTA DE `POST /v1/predict`

Campo	Contenido
`decision`	`ACT`, `ANGLE_UNCERTIFIED`, `REVIEW`, `UNKNOWN` o `BAD_CAPTURE`
`reasons`	Reglas que impidieron actuar, en texto
`robot_command`	Orden para el robot: `action` y, si es `PICK`, producto y ángulo
`object_id`, `class_confidence`, `novelty`	Producto, confianza del clasificador y distancia a la vista conocida más cercana
`angle_deg`, `angle_credible_width_deg`	Ángulo y ancho del conjunto creíble del 90 %
`topk_pose`, `topk_appearance`	Vistas similares por ángulo y por apariencia
`quality`	Validación de la captura: tamaño, fracción de objeto y brillo
`request_id`, `model_version`, `latency_ms`, `line_alert`	Por petición: identificador, versión del modelo, tiempo y alerta de la línea

B. API del asistente

El asistente tiene su propia API, que es la que consume la interfaz web (Tabla X). El acceso empieza en `POST /api/login`, que devuelve un token de sesión; las demás llamadas lo envían en la cabecera `Authorization`. `POST /api/chat` recibe el mensaje y las imágenes, si las hay, y devuelve el texto de la respuesta, el motivo por el que terminó el turno, los pasos ejecutados con su resultado y su latencia, la evidencia de las inspecciones, las solicitudes creadas y el consumo de tokens. La decisión sobre una operación de riesgo tiene su propia ruta, que el modelo de lenguaje no puede llamar. La documentación interactiva también está en `/docs`, y los contratos de las 22 *tools* en `doc/openapi/tools.json`.

TABLA X
API DEL ASISTENTE

Método y ruta	Qué hace
`POST /api/login`	Inicia sesión con un usuario; devuelve el token y las *tools* de su rol
`POST /api/chat`	Un turno: mensaje e imágenes; devuelve respuesta, pasos, evidencia y consumo
`GET /api/approvals`	Solicitudes pendientes y recientes
`POST /api/approvals/{id}/decision`	Aprueba o rechaza una operación de riesgo. No es una *tool*
`GET /api/audit`	Registro de auditoría, para los roles autorizados
`GET /api/tools` `GET /api/lines`	Catálogo de *tools*; estado de las líneas y alerta de captura
`GET /api/sessions/{id}/messages` `GET /api/images/{id}`	Historial de la sesión e imágenes subidas

C. Despliegue

El sistema no se desplegó en ninguna plataforma: no hay servidor ni servicio en la nube. Se ejecuta en un equipo local, con un comando o con Docker Compose, que levanta dos contenedores: el servicio de visión en el puerto 8001 y el asistente en el 8002. Para dar acceso desde fuera durante la demostración se usa un túnel de ngrok hacia el puerto del asistente, que publica una dirección HTTPS temporal mientras el equipo está encendido. La API de visión no se expone: el asistente la consume por la red local.

Esta forma de exposición tiene límites. La dirección cambia en cada sesión y la disponibilidad depende de un solo equipo. Con una dirección pública hay que activar el código de acceso del asistente y desactivar la inyección de fallos del servicio de visión. La arquitectura objetivo en Azure, con la inferencia junto a la línea, está diseñada en el repositorio y no desplegada.

X. Conclusiones

El trabajo deja cinco conclusiones. Primera, en este problema la dificultad no está en clasificar sino en estimar el ángulo cuando faltan vistas y en saber abstenerse; medirlo antes de construir orientó el resto. Segunda, la representación pesa más que el clasificador, y un criterio fijado de antemano eligió un modelo distinto del que se esperaba. Tercera, tratar las vistas de un objeto como un lazo da un ángulo continuo y, además, una distribución de la que salen dos medidas de incertidumbre que se complementan: el ancho detecta los objetos que se ven igual desde cualquier lado y la ambigüedad detecta el segundo candidato a media vuelta. Cuarta, un control determinista calibrado sin tocar la prueba reduce las acciones inseguras del 4,1 % al 0,7 % en A y las elimina en B, C y D, y la referencia moderna no mejora las medidas de las que depende la seguridad. Quinta, en el asistente lo útil del modelo de lenguaje es entender y explicar; las decisiones que importan quedan en el código y en las personas.

Las limitaciones son concretas. El objetivo de riesgo no se alcanza del todo en prueba (0,79 % frente a 0,5 %). Los datos son de laboratorio: con el objeto desplazado 8 píxeles, las acciones inseguras suben al 10,5 %, y un cambio de brillo hace rechazar producto conocido. Las correcciones están diagnosticadas y no implementadas: localizar y centrar el objeto antes de describirlo, normalizar el brillo, y una regla que compare la distancia de la imagen a su lazo. La evaluación del asistente no tiene conjunto reservado, y la planta es simulada. El sistema no está desplegado en ninguna plataforma: corre en un equipo local y se expone con un túnel temporal. Antes de un piloto habría que recalibrar todo con imágenes de la propia línea.

TABLA XI
Implementación

Herramienta	Uso
Python	Lenguaje de todo el proyecto
scikit-learn, NumPy	Características, clasificadores, PCA y lazos
MLflow	Registro de experimentos y de modelos
FastAPI	Servicio de visión y servicio del asistente
SQLite	Planta simulada, sesiones y auditoría
Gemini	Modelo de lenguaje del asistente
Docker	Despliegue local con un comando
ngrok	Túnel HTTPS temporal para la demostración
GitHub	Repositorio con el código y los reportes

Iconos de Simple Icons (licencia CC0). Las marcas pertenecen a sus titulares.

Referencias

[1] S. A. Nene, S. K. Nayar y H. Murase, «Columbia Object Image Library (COIL-100)», Columbia Univ., Nueva York, inf. téc. CUCS-006-96, 1996.

[2] H. Murase y S. K. Nayar, «Visual learning and recognition of 3-D objects from appearance», *Int. J. Comput. Vis.*, vol. 14, n.º 1, pp. 5–24, 1995.

[3] N. Dalal y B. Triggs, «Histograms of oriented gradients for human detection», en *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2005, pp. 886–893.

[4] T. Ojala, M. Pietikäinen y T. Mäenpää, «Multiresolution gray-scale and rotation invariant texture classification with local binary patterns», *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, n.º 7, pp. 971–987, 2002.

[5] L. Breiman, «Random forests», *Mach. Learn.*, vol. 45, n.º 1, pp. 5–32, 2001.

[6] C. Cortes y V. Vapnik, «Support-vector networks», *Mach. Learn.*, vol. 20, n.º 3, pp. 273–297, 1995.

[7] M. Oquab *et al.*, «DINOv2: Learning robust visual features without supervision», *Trans. Mach. Learn. Res.*, 2024.

[8] S. Yao *et al.*, «ReAct: Synergizing reasoning and acting in language models», en *Proc. Int. Conf. Learn. Represent.*, 2023.

[9] S. Robertson y H. Zaragoza, «The probabilistic relevance framework: BM25 and beyond», *Found. Trends Inf. Retr.*, vol. 3, n.º 4, pp. 333–389, 2009.

[10] F. Pedregosa *et al.*, «Scikit-learn: Machine learning in Python», *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[11] Y. Geifman y R. El-Yaniv, «Selective classification for deep neural networks», en *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.

[12] T. Cover y P. Hart, «Nearest neighbor pattern classification», *IEEE Trans. Inf. Theory*, vol. 13, n.º 1, pp. 21–27, 1967.